cc-review

Many reviewers, one verdict

Multi-agent code review for Claude Code, Codex and OpenCode

Roland Huß

A review starts now and finishes during the talk

```
/cc-review:review --pr <n> --profile thorough
```

Gate: PASS (after fix round 1)

Reviewer	Found	Fixed	Remaining	Status
Correctness	4	4	0	completed
Production Readiness	3	2	1	completed
CodeRabbit	3	2	1	completed
Greptile	–	–	–	skipped (not logged in)

Illustrative result: real runs look different

Most AI review comments get ignored

56%

of CodeRabbit suggestions rejected

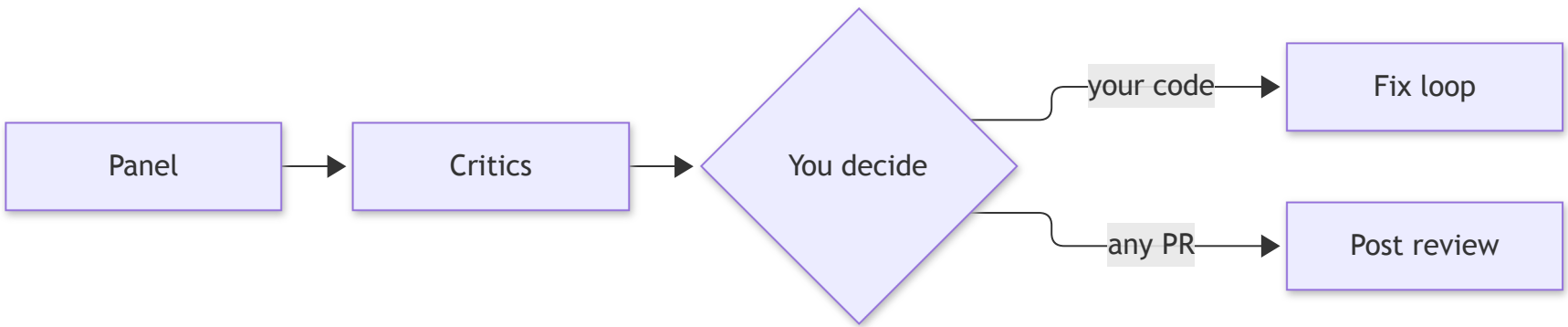
1-19%

of AI comments addressed
(humans: ~60%)

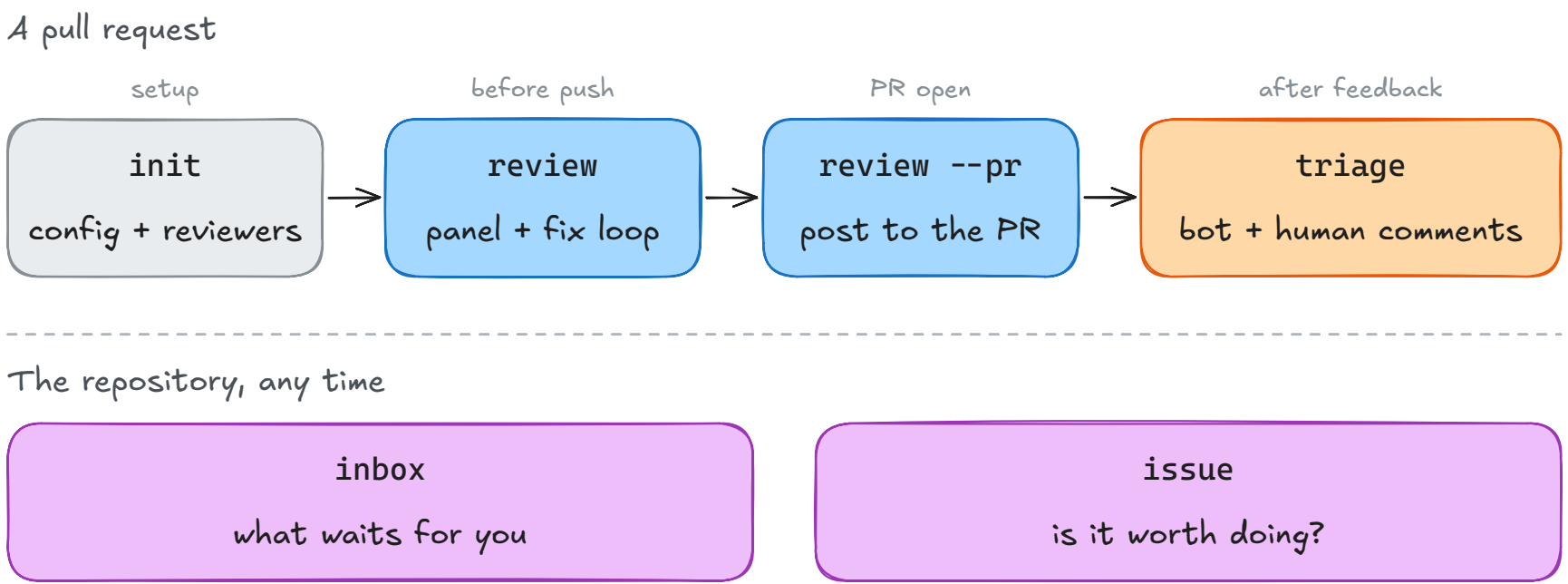
arXiv 2607.03316 (31k reviews, 239 repos) · arXiv 2508.18771 (16 tools, 178 repos, ~22k comments)

cc-review runs a panel, then makes it argue

- **Focused reviewers** plus external tools, in parallel
- **Two critics** filter before you see anything
- **You decide**: fix your own code, or post to the PR
- **Local first**: before you push



Five commands: one PR flow, one repo view



The next slides go deep on `review` · `triage`, `inbox` and `issue` follow as a live demo after the results

Two commands to install

```
/plugin marketplace add rhuss/cc-rhuss-marketplace
/plugin install cc-review@cc-rhuss-marketplace
```

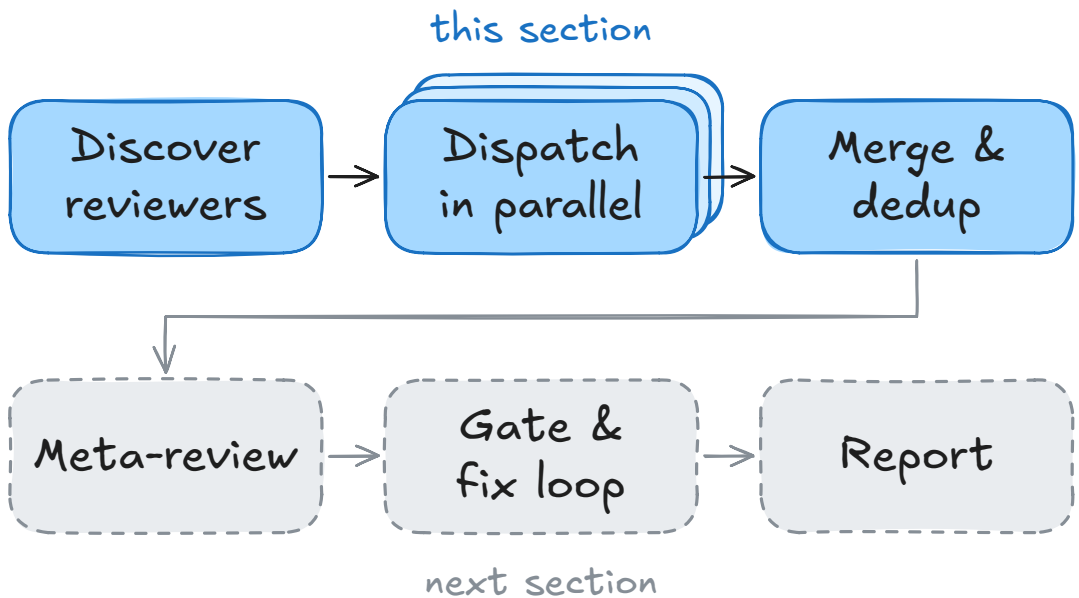
- **Codex**: `codex plugin install` from a clone
- **OpenCode**: `review/opencode/install.sh`
- Needs `jq`, `yq`, `gh`, Python 3.9+

Three ways to run your first review

```
/cc-review:review                    # your branch vs main
/cc-review:review --pr 42            # a PR, in its own worktree
/cc-review:review --profile quick    # cheap: 2 reviewers, 1 round
```

- Profiles: `quick` · `thorough` · `ci`
- Project config: `.cc-review/config.yml`
- Project knowledge: `.cc-review/review-hints.md`

What is running right now



Discover: no hard-coded list

Dispatch: subagents and CLIs together

Merge: same spot, same category, one finding

Six lanes and four outside voices

- **Correctness**
- **Architecture**
- **Security**
- **Production**
- **Test Quality**
- **Goal Alignment**

External

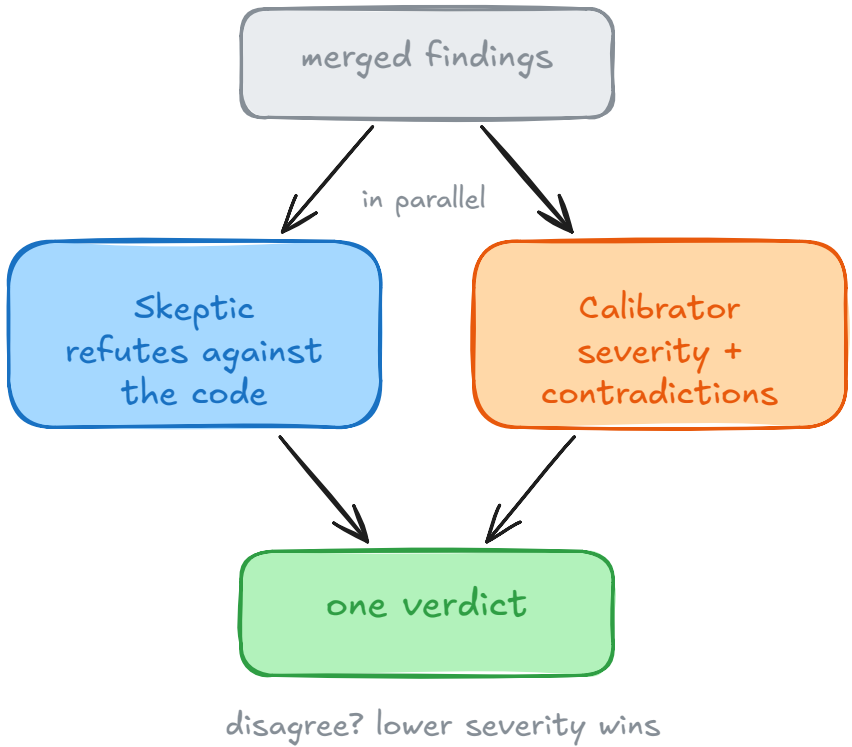
CodeRabbit · Codex
Copilot · Greptile (opt-in)

Opt-in: Gator, principal-engineer persona from NVIDIA OpenShell

"Judge what was built, not what was intended."

- No praise, no quota: **zero findings is valid**
- Distrust comments and commit messages
- No git log, no plan documents
- Confidence ⩾ 70, with file, line, impact, fix

Two critics check every finding



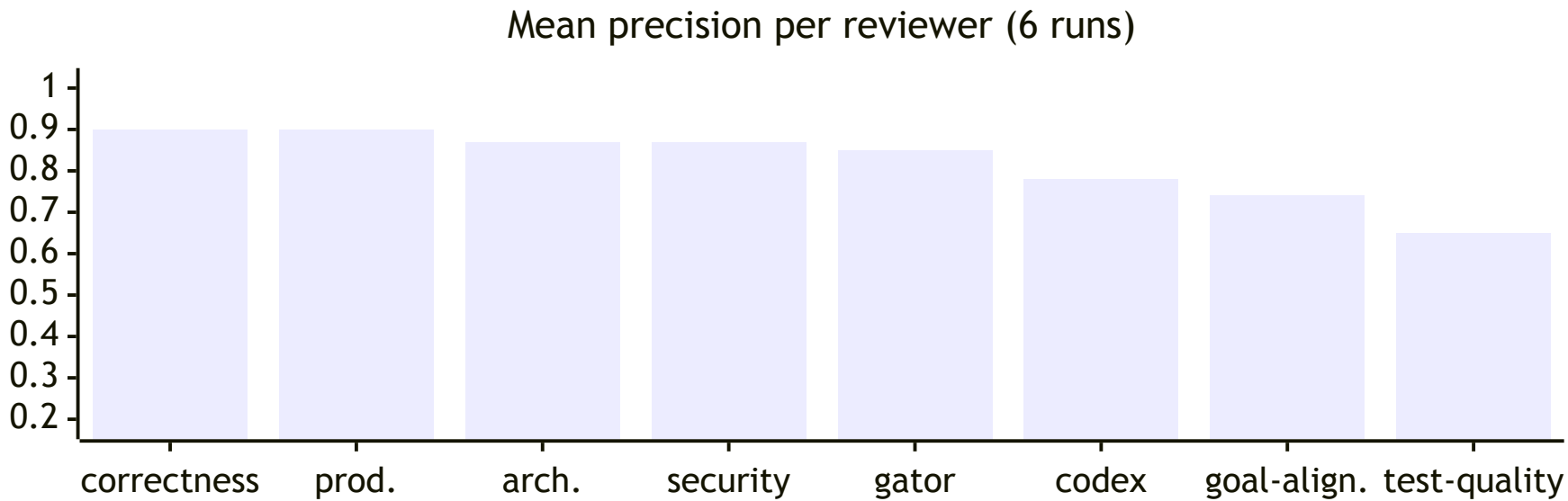
13%

of findings downgraded by the Calibrator
(6 runs, 128 findings)

0

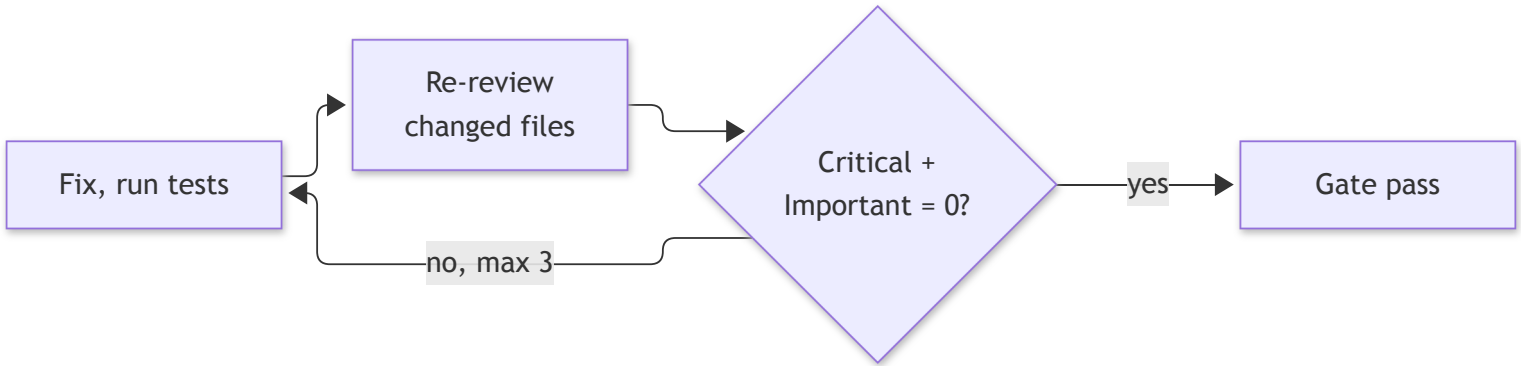
findings removed by the Skeptic
(10 marked weak)

The panel grades itself on every run



- **test-quality** lowest in every run: first candidate for review hints or `--no-test-quality`

On your own code: fix, test, re-review



82%

of Critical + Important fixed
(547 of 670, 140 reports)

- You pick: all fixes, or gate only
- Failing test → **Critical** finding
- Others' PRs: post, never push

On real PRs, two thirds of the findings get fixed

208
findings, 20 PRs

64%
addressed

2%
withdrawn or rejected

- Double percent-decoding of OTLP headers (Codex, #5082)
- Credentials in startup log (security, #5083)
- Trace ID mismatch CodeRabbit missed (gator, #4789)

NVIDIA/OpenShell, 30 cc-review reviews · Important findings: 87% addressed

Triage, inbox and issue handle the rest

`triage`

Bot and human comments; drafts replies

`inbox`

PRs, issues, Discussions by who acts next

`issue`

Need, fit, value; reply draft

More on this in the demo

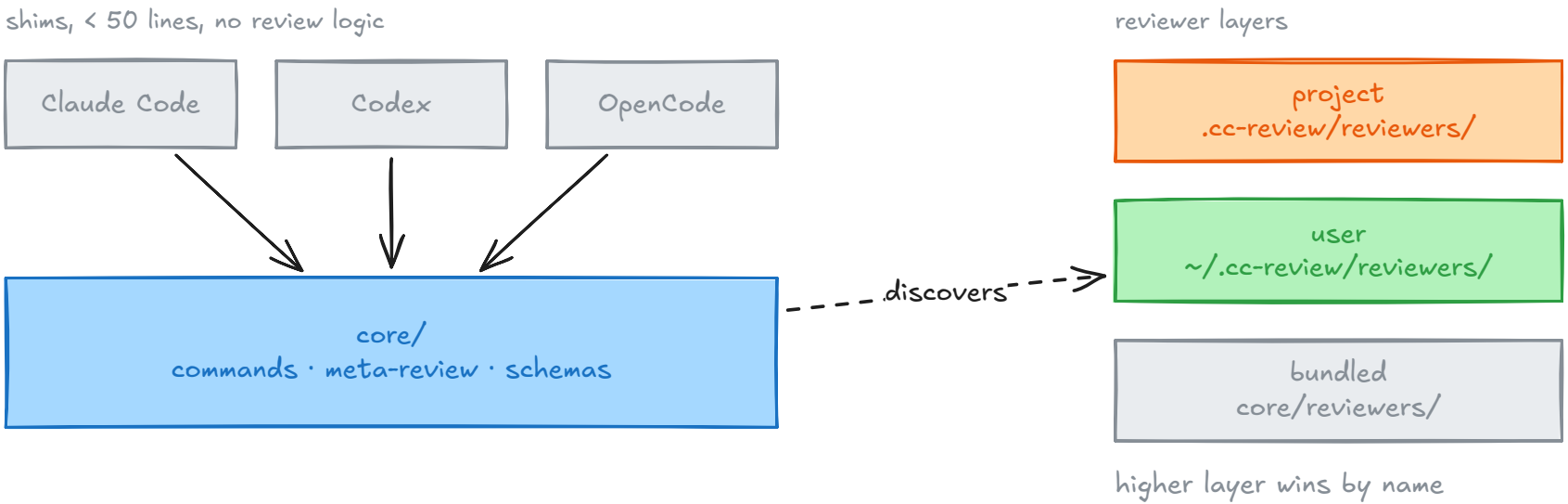
Review many PRs at once, and only what changed

```
/cc-review:review --bulk
/cc-review:review --bulk --resume --unattended
```

Selected	review requested from you or your team, or new commits since your last cc-review
Or pick	`--all-open` · `--search "label:bug"` · `--prs 101,104`
Skipped	drafts, PRs already reviewed at their current commit
Then	all PRs reviewed first, one decision table, only then any action

- Follow-ups: **only the delta**, earlier findings verified

One core, three agents, reviewers found at run time



A reviewer is one Markdown file

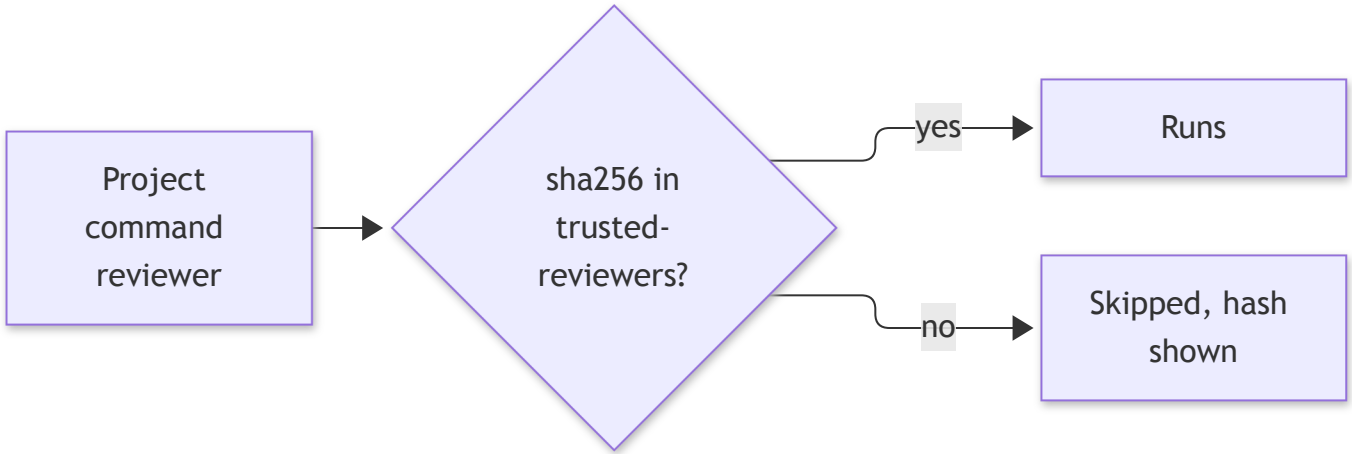
```
---
name: team-rules
display_name: "Team Rules"
kind: prompt
category: conventions
---
You are the TEAM RULES REVIEW AGENT.
YOUR SCOPE: our API error format, logging fields,
feature flags. Stay in your lane.
```

- Save as `.cc-review/reviewers/team-rules.md`
- Next run: in the status table; `--only team-rules`

Wrapping an external tool takes a frontmatter block

```yaml
name: greptile
display_name: "Greptile"
kind: command
enabled: false
detect: "command -v greptile"
auth_check: "greptile whoami >/dev/null && greptile init --json >/dev/null"
invoke: "greptile review --json --agent -b {base_branch}"
timeout_seconds: 540
severity_map:
  P0: Critical
  P1: Important
  P2: Minor
output:
  format: parser
  parser: parse.sh
```

Project reviewers run only after you trust them



- A change **cannot judge itself**
- Fork checkouts: command reviewers skipped

Three takeaways



Diversity wins

More agents, more vendors, fewer blind spots



Critics

Every finding checked against the code before you see it



Your code

On your code: fix, test, re-review

Try it!

- Install from `rhuss/cc-rhuss-marketplace`
- Run `/cc-review:review --profile quick` on your next branch
- Write one reviewer for a rule your team repeats
- Read the full guide: `docs/GUIDE.md` (also as PDF)
- These slides: PDF on GitHub



github.com/rhuss/cc-review

Backup: most review tools are one vendor's single opinion

Tool	Delivery	Multi-pass	Local
CodeRabbit	PR bot, CLI	review + verify	yes
Copilot review	PR bot, IDE	single	IDE
Greptile	PR bot, CLI	n/a	committed
Codex	CLI, cloud	single	yes
Claude Code Review	GitHub App	team + verify	no
Cursor Bugbot	PR bot	passes + validator	yes

Backup: multi-pass review with verification is the industry pattern

- Bugbot: resolution rate 52% → 70%+ (vendor)
- Anthropic: <1% findings marked incorrect (vendor)
- Cubic: 51% fewer false positives (vendor)
- CodeRabbit field study: 56% rejected (independent)

Backup: every finding follows one schema

```
{
  "id": "FINDING-8", "severity": "Important", "confidence": 95,
  "file": "review/core/scripts/inbox.py", "line_start": 290,
  "category": "correctness", "source_agent": "correctness",
  "also_reported_by": [], "verdict": "confirmed"
}
```

- Severities: Critical · Important · Minor · Notable
- `severity_map` applied once, at merge

Backup: config resolution and key flags

CLI › `--config` › `--profile` › project › user › defaults

```
--pr <n>  --full  --spec <file>  --profile <name>
--only a,b  --no-<name>  --no-external  --no-fix
--max-rounds <n>  --no-meta-review  --bulk  --resume
```

Backup: cost scales with the panel you choose

- Prompt reviewers: one subagent each
- Meta-review: two more subagents (⩾5 findings)
- External tools: their own credits
- `quick`: 2 reviewers, 1 round